

Course No. CS F407

Course Name. Artificial Intelligence

Your ID No: _____

Name: _____

Date: March 3, 2025

Type: Closed Book

Total Marks: 60

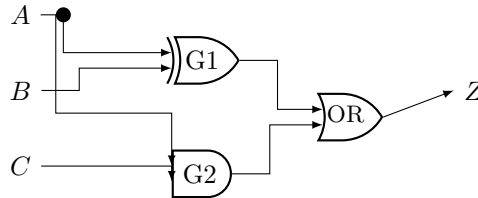
Time: 90 Mins

General Instructions:

- This Question Paper contains two parts: **Part-(A)** and **Part-(B)**. Answer the questions of Part-(A) in a separately dedicated Answer Sheet.
- Answers to all questions of **Part-(B)** must be written in the answer booklet provided separately. **Answers to all the questions must be in serial order in the answer book, with all sub-parts of each question together.**
- Only answers written in ink (**NOT in pencil**) will be evaluated.
- **Separate your Answer Sheet for Part-(A) and the Answer Booklet for Part-(B)** while submitting at the end of the examination.
- All necessary working must be shown in the answer sheet itself. Merely writing the answers is not sufficient.
- Write your Name and ID No in all pages of Answer Sheets / Booklets. **(Best wishes)!!**

Part - (A)

1. **Logic Statement:** ‘If $G2$ is an AND gate, then its output is 1 if and only if the output of $G1$ is 1 *and* terminal input C is 1.’



Using the predicates and functions (e.g., $Gate(g)$, $Type(g)$, $In(n, g)$, $Out(1, g)$, $Signal(t)$ etc.), which one of the following is the correct first-order logic formula for the statement above? **(2 Marks)**

- A. $\forall g \left(Gate(g) \wedge Type(g) = AND \rightarrow [Signal(Out(1, g)) = 1 \Leftrightarrow Signal(In(1, g)) = 1 \vee Signal(In(2, g)) = 1] \right)$.
- B. $\forall g \left(Gate(g) \wedge Type(g) = AND \rightarrow [Signal(Out(1, g)) = 1 \Leftrightarrow Signal(In(1, g)) = 1 \wedge Signal(In(2, g)) = 1] \right)$
- C. $\exists g \left(Gate(g) \wedge Type(g) = AND \wedge [Signal(Out(1, g)) = 0 \Leftrightarrow Signal(In(1, g)) = 0] \right)$
- D. $\forall g \left(Gate(g) \wedge Type(g) = AND \rightarrow [Signal(Out(1, g)) = 1 \Rightarrow Signal(In(1, g)) = 1] \right)$

Solution and Explanation: If $G2$ is an AND gate, then its output is 1 if and only if the output of $G1$ is 1 and the terminal input C is 1. Then, we can use the following Predicate and Functions:

- $Gate(g)$: True if g is a gate.
- $Type(g) = AND$: Specifies that gate g is of type AND.
- $Out(n, g)$: The n -th output terminal of gate g .
- $Signal(x)$: The signal value at terminal or wire x .
- $In(n, g)$: The n -th input terminal of gate g (used in some formalizations).

Since the statement focuses on $G2$ and specifically its inputs as $Out(1, G1)$ and C , we do not need a universal quantifier over all gates g ; we only specify the condition for $G2$ being an AND gate.

Translating the Statement into First-Order Logic

1. "If $G2$ is an AND gate" translates to: $Gate(G2) \wedge (Type(G2) = AND)$.
2. "then its output is 1 if and only if the output of $G1$ is 1 and C is 1" translates to:

$$Signal(Out(1, G2)) = 1 \iff (Signal(Out(1, G1)) = 1 \wedge Signal(C) = 1).$$

3. Combine with implication:

$$(Gate(G2) \wedge (Type(G2) = AND)) \rightarrow (Signal(Out(1, G2)) = 1 \iff (Signal(Out(1, G1)) = 1 \wedge Signal(C) = 1)).$$

Final First-Order Logic Formula:

$$(Gate(G2) \wedge (Type(G2) = AND)) \rightarrow (Signal(Out(1, G2)) = 1 \iff (Signal(Out(1, G1)) = 1 \wedge Signal(C) = 1)).$$

An AND gate outputs 1 precisely when *all* of its inputs are 1. Here, $G2$ has two inputs: Input 1 = $Out(1, G1)$ and Input 2 = C . Thus, for $G2$ to output 1, we need $Signal(Out(1, G1)) = 1$ and $Signal(C) = 1$. The equivalence (" $\iff$ ") between " $G2$'s output is 1" and " $G1$'s output and C are both 1", but only in the case where $G2$ is of type AND. **Answer (Correct MCQ Choice): B**

2. Suppose $P(\text{Disease}) = 0.01$ and $P(\text{TestPositive} \mid \text{Disease}) = 0.99$. If the overall chance of a positive test in the population is 0.05, then $P(\text{Disease} \mid \text{TestPositive})$ is: **(2 Marks)**

- A. $\frac{0.99 \times 0.05}{0.01}$ B. $\frac{0.05 \times 0.99}{0.01}$ C. $\frac{0.99 \times 0.01}{0.05}$ D. $\frac{0.01 \times 0.99}{1 - 0.05}$

Solution and Explanation. We are given: $P(\text{Disease}) = 0.01$, $P(\text{TestPositive} \mid \text{Disease}) = 0.99$, and the overall probability of a positive test in the population is $P(\text{TestPositive}) = 0.05$. We want to find $P(\text{Disease} \mid \text{TestPositive})$.

Applying Bayes' Theorem $P(\text{Disease} \mid \text{TestPositive}) = \frac{P(\text{TestPositive} \mid \text{Disease}) \times P(\text{Disease})}{P(\text{TestPositive})}$.

Substitute the values: $\frac{(0.99) \times (0.01)}{0.05} = \frac{0.0099}{0.05} = 0.198$.

Thus, $P(\text{Disease} \mid \text{TestPositive}) = 0.198$ (i.e., 19.8%). Therefore, the correct choice is

$$\text{(C)} \frac{0.99 \times 0.01}{0.05} = 0.198.$$

3. Which term is commonly referred to as the *prior probability* in Bayes' theorem $P(\text{Cause} \mid \text{Effect}) = \frac{P(\text{Effect} \mid \text{Cause}) P(\text{Cause})}{P(\text{Effect})}$? **(2 Marks)**

A. $P(\text{Cause} \mid \text{Effect})$ B. $P(\text{Effect} \mid \text{Cause})$ C. $P(\text{Cause})$ D. $P(\text{Effect})$

Solution and Explanation. The *prior probability* refers to the probability of the event (here, “Cause”) prior to (i.e., without) taking into account the new evidence (“Effect”). In the formula $P(\text{Cause} \mid \text{Effect}) = \frac{P(\text{Effect} \mid \text{Cause}) P(\text{Cause})}{P(\text{Effect})}$, the term $P(\text{Cause})$ is called the *prior*. Therefore, the correct choice is

$$\text{(C)} P(\text{Cause}).$$

4. Consider a *logistic model* for the probability that a customer buys, given a continuous cost c :

$$P(\text{buys} \mid c) = ?$$

Which of the following functional forms corresponds to a *logistic* (a.k.a. logit) model with location parameter μ and scale parameter σ ? **(2 Marks)**

A. $\Phi\left(\frac{c - \mu}{\sigma}\right)$ (*normal CDF*) B. $\frac{1}{1 + e^{-\frac{c - \mu}{\sigma}}}$ C. $e^{-(c - \mu)/\sigma}$ D. $1 - \Phi\left(\frac{c - \mu}{\sigma}\right)$

Solution and Explanation. A *logistic* (or “logit”) model for a probability typically has the form $P(\text{buys} \mid c) = \frac{1}{1 + e^{-z}}$, where $z = \frac{c - \mu}{\sigma}$.

- μ is often called the “location” parameter (it shifts the midpoint of the logistic curve).
- σ is the “scale” parameter (it stretches or shrinks the curve).

As c increases (for $\sigma > 0$), z increases, so e^{-z} decreases and thus $P(\text{buys} \mid c)$ moves from near 0 to near 1 in a smooth “S-shaped” curve. Only option (B) has the characteristic “ $\frac{1}{1 + e^{(\cdot)}}$ ” form of a logistic function. Note that in the *most common* parametrization, we often see $\frac{1}{1 + e^{-\frac{c - \mu}{\sigma}}}$. **Option (B)** uses $+\frac{c - \mu}{\sigma}$ in the exponent instead of $-\frac{c - \mu}{\sigma}$, but it is still a valid logistic functional form. Indeed,

$$\frac{1}{1 + e^{\frac{c - \mu}{\sigma}}} = \frac{1}{1 + e^{-\left(\frac{\mu - c}{\sigma}\right)}},$$

which is simply the logistic function mirrored in terms of c . Regardless of the sign convention, the “ $1/(1 + e^{\cdot})$ ” shape is unique to the logistic family among these options.

The correct choice (logistic model) is **(B)**.

5. Suppose the cost C is a continuous random variable with a *normal* prior, $C \sim \mathcal{N}(\mu, \sigma^2)$, and the likelihood of observing buys is given by the logistic function $P(\text{buys} \mid c) = \frac{1}{1 + e^{-(c - \mu)/\sigma}}$. By Bayes' Rule, the *posterior* density of C given buys is (up to a normalization constant):

- A. $\mathcal{N}(c \mid \mu, \sigma^2) + \text{logistic}(c)$ B. $\text{Uniform}(0, 1) \times \text{logistic}(c)$ C. $\mathcal{N}(c \mid \mu, \sigma^2) \times \text{logistic}(c)$ D. $\mathcal{N}(c \mid 0, 1) \div \text{logistic}(c)$ **(2 Marks)**

Solution and Explanation By Bayes' rule, the *posterior* distribution for C given that “buys” was observed is proportional to the product of: (prior) $\times$ (likelihood).

Prior: $p(c) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(c-\mu)^2}{2\sigma^2}\right)$. Likelihood: $P(\text{buys} \mid c) = \frac{1}{1+e^{-\frac{c-\mu}{\sigma}}}$. Therefore, Posterior (up to normalization):

$$p(c \mid \text{buys}) \propto p(c) \times P(\text{buys} \mid c) = \underbrace{\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(c-\mu)^2}{2\sigma^2}\right)}_{\text{Normal}(c; \mu, \sigma^2)} \times \underbrace{\frac{1}{1+e^{-\frac{c-\mu}{\sigma}}}}_{\text{logistic}(c)}.$$

Thus, the posterior density, among the given options, the correct choice is (up to a constant factor):

$$\text{(C)} \quad \mathcal{N}(c \mid \mu, \sigma^2) \times \text{logistic}(c).$$

6. A simple translational embedding model, *TransE*, for a triple (s, p, o) , often enforces which energy/score function? **(2 Marks)**
- A. $\|\mathbf{s} + \mathbf{p} - \mathbf{o}\|$ to be small for valid triples.
 B. $\mathbf{s} \cdot \mathbf{p} - \mathbf{o}$ to be large for valid triples.
 C. $\|\mathbf{s} \odot \mathbf{p} + \mathbf{o}\|$ to be large for valid triples.
 D. $|\mathbf{s}| + |\mathbf{p}| + |\mathbf{o}| = 0$.

Solution and Explanation.

In *TransE*, the basic assumption is $\mathbf{s} + \mathbf{p} \approx \mathbf{o}$ for a true (valid) fact (s, p, o) . Hence the model tries to minimize the distance $\|\mathbf{s} + \mathbf{p} - \mathbf{o}\|$ for positive triples in **Option (B)**.

7. Which of the following is *not* typically considered a **downstream task** on knowledge graphs? **(2 Marks)**
- A. Link prediction B. Entity classification C. Similarity computation of entities
 D. Matrix factorization of node adjacency for big data *only*

Solution and Explanation:

While link prediction, entity classification, and entity similarity are common tasks in KGs, direct “matrix factorization of node adjacency for big data only” in **Option (D)** is *too specific* (and not necessarily a direct KG task).

8. A heuristic $h(n)$ is called *admissible* if, for every node n , it satisfies: $h(n) \leq ?$ Which of the following best completes this inequality? **(2 Marks)**
- A. $h(n) \leq 2g(n)$
 B. $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal
 C. $h(n) \leq f(n)$
 D. $h(n) \leq 0$

Solution and Explanation. A heuristic $h(n)$ is *admissible* if it *never overestimates* the true cost to reach the goal. Formally, the **Option (B)** $h(n) \leq h^*(n)$, where $h^*(n)$ denotes the actual optimal cost from n to the goal.

9. A^* search is guaranteed to find an *optimal* path to the goal if: (2 Marks)
- A. The state space is finite, and $h(n)$ is consistent
 - B. The heuristic $h(n)$ is admissible, and the search space is a directed acyclic graph (DAG)
 - C. The heuristic $h(n)$ is admissible, and the search uses tree search (no repeated states)
 - D. The heuristic $h(n)$ is admissible, and the branching factor is constant

Solution and Explanation. A^* is guaranteed to find an optimal path with *admissible* $h(n)$ if it uses a *graph search* (i.e., detecting and avoiding re-expanding higher f -cost duplicates) or if h is *consistent* even under tree search. Therefore, **Option (A)** is correct.

10. Which of the following best describes the *fringe* management of Breadth-First Search (BFS)? (2 Marks)
- A. BFS is an informed search strategy, and it treats the *fringe* as a Last-In-First-Out (LIFO) stack
 - B. It always expands the node with the lowest path cost $g(n)$
 - C. BFS is an uninformed search strategy, and it treats the fringe as a First-In-First-Out (FIFO) queue
 - D. BFS is an informed search strategy, and it treats the fringe as a FIFO queue

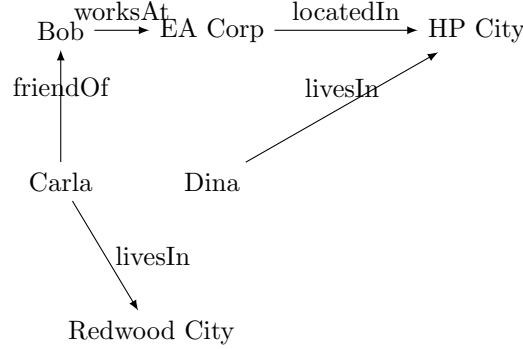
Solution and Explanation. The Breadth-First Search enqueues new successors at the *end* of the queue, thus it is a FIFO structure as stated in **Option (C)**.

Part - (B)

1. Below is a knowledge graph $\mathcal{G}$ with six entities: $\mathcal{E} = \{\text{Bob, Carla, Dina, EA Corp, Redwood City, HP City}\}$ and four relations: $\mathcal{R} = \{\text{works At, friend Of, lives In, located In}\}$. The known *positive* (true) triples are:
- $\mathcal{Q}^+ = \{(\text{Bob, works At, EA Corp}), (\text{Carla, friend Of, Bob}), (\text{Carla, lives In, Redwood City}), (\text{EA Corp, located In, HP City}), (\text{Dina, lives In, HP City})\}$.
- (a) Draw out the diagram of the Knowledge-Graph $\mathcal{G}$. (2 Marks)
 - (b) Formulate how to generate *synthetic negative* triplets by “corrupting” the subject or object in each of the above positive triples under a *Local Closed World* assumption. Also, draw the diagram of the corrupted knowledge graph $\hat{\mathcal{G}}$. (4 Marks)
 - (c) Suppose you train the embedding model of $\mathcal{G}$ on the above data, and then you test on the three true triplets: $\mathcal{Q}^{test} = \{(\text{Bob, works At, EA Corp}), (\text{Carla, friend Of, Bob}), (\text{Dina, lives In, HP City})\}$. Calculate the Mean Rank (MR) and Mean Reciprocal Rank (MRR) by assuming the model’s predicted *ranks* for these three query triples 2, 5, and 1, respectively. (4 Marks)

Q1. Solution and Explanation

(a) **Diagram of the Knowledge Graph G** A simple pictorial representation of G can be drawn as a directed graph. For instance, one could use `tikz` as follows:



Here, each directed edge is labeled by the corresponding relation from $\mathcal{R}$.

(b) Generating Synthetic Negative Triplets

Under a *Local Closed World* assumption, any triple not explicitly stated as *true* in Q^+ is treated as *false*. To generate synthetic negative triplets, we often use the standard “corruption” approach:

- For each positive triple (s, p, o) ,
- Replace s with another entity $s' \in E$ (*different from s*) that does not make (s', p, o) a known positive triple, **or**
- Replace o with another entity $\hat{o} \in E$ (*different from o*) that does not make $(s, p, \hat{o})$ a known positive triple.

The formulation below is used to corrupt the existing Knowledge Graph

Example. From the positive triple $(\text{Bob}, \text{worksAt}, \text{EA Corp})$, we can create corrupted triples such as: $(\text{Carla}, \text{worksAt}, \text{EA Corp})$, $(\text{Dina}, \text{worksAt}, \text{EA Corp})$, $(\text{Bob}, \text{worksAt}, \text{Redwood City})$, $(\text{Bob}, \text{worksAt}, \text{HP City})$, $\dots$, provided none of these appear in Q^+ . One can similarly corrupt the other four positive triples to get a set of negative triples. If we imagine a larger “combined” diagram—call it $\hat{G}$ —it would contain the original positive edges plus all these extra “corrupted” edges that the model should learn to give a low likelihood score. Visually, it would have many more edges (all labeled as negative examples) emanating from each relation.

(c) Mean Rank (MR) and Mean Reciprocal Rank (MRR)

We train a knowledge-graph embedding model on the original data Q^+ (and the synthetic negatives). Then we test on the following three *true* (held-out) triples:

$$Q_{\text{test}} = \{(\text{Bob}, \text{worksAt}, \text{EA Corp}), (\text{Carla}, \text{friendOf}, \text{Bob}), (\text{Dina}, \text{livesIn}, \text{HP City})\}.$$

Assume the model produces predicted *ranks* for these three query triples of: (2, 5, 1). Here, rank 1 means the triple is the model's top prediction among all possible candidates; rank 2 means it is second place, and so on.

Mean Rank (MR). By definition, $MR = \frac{\sum_{i=1}^{|Q_{\text{test}}|} \text{Rank}(i)}{|Q_{\text{test}}|}$. In this case, $|Q_{\text{test}}| = 3$, and the sum of ranks is $2 + 5 + 1 = 8$. Thus, $MR = \frac{8}{3} \approx 2.67$.

Mean Reciprocal Rank (MRR). The reciprocal rank of a triple whose rank is r is simply $\frac{1}{r}$. Hence:

$$MRR = \frac{1}{|Q_{\text{test}}|} \sum_{i=1}^3 \frac{1}{\text{Rank}(i)} = \frac{(\frac{1}{2} + \frac{1}{5} + 1)}{3} = \frac{(0.5 + 0.2 + 1.0)}{3} = \frac{1.7}{3} = 0.5666 \dots$$

Thus, the final numeric values are: $MR \approx 2.67$, $MRR \approx 0.57$.

2. We have a sliding **8-puzzle** on a 3×3 board. The *blank* (denoted by $\square$) can move up/down/left/right if there is room. The *goal state* is the **diagonal-major order**, defined as follows:

Initial State $\mathcal{S}^{init}$

1	5	7
4	$\square$	2
8	3	6

$\xrightarrow{\text{Solve}}$

Goal State $\mathcal{S}^{goal}$

1	4	7
2	5	8
3	6	$\square$

- (a) *Propose an iterative algorithm* (e.g., iterative deepening) in pseudo-code that transforms the $\mathcal{S}^{init}$ to $\mathcal{S}^{goal}$. Your algorithm must specify the following: **(6 Marks)**
- *State Representation* as 3×3 matrix $\mathcal{S}$ with entries $\in \{\mathcal{S}_{ij} \mid \forall (i, j) 1 \leq i, j \leq 3\}$ is represented and how *moves* (actions) are generated,
 - The *depth limit*, iteration scheme, action as swap between $\mathcal{S}_{ij}$ and a blank $\mathcal{S}_{bk}$ assuming $\mathcal{S}_{bk} = 0$.
 - The condition to detect the *goal state*.
 - Each slide of the blank is 1 unit of effort, so the path cost is the number of moves $g(n) = n$.
- (b) *Demonstrate the **first three** expansions* (successor generations) from the initial state according to your iterative method. Draw a small *tree/graph* structure or a step-by-step tabular form to show the newly generated states. **(4 Marks)**

Q2. Solution: IDS for the 8-Puzzle

(a) **Proposed Iterative Algorithm** We wish to solve the sliding 8-puzzle from an initial state $\mathcal{S}_{init}$ to the goal state $\mathcal{S}_{goal}$, which is defined in “diagonal-major”

order (where $\square$ indicates the blank tile.):

$$S_{\text{goal}} = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & \square \end{bmatrix}$$

1. State Representation A state S can be represented by a 3×3 matrix of integers:

$$S = \begin{bmatrix} S_{11} & S_{12} & S_{13} \\ S_{21} & S_{22} & S_{23} \\ S_{31} & S_{32} & S_{33} \end{bmatrix},$$

where each entry $S_{ij} \in \{0, 1, 2, \dots, 8\}$ with 0 (or $\square$) denoting the blank tile. Equivalently, we could store the state as a one-dimensional array of length 9, but the concept is the same.

2. Generating Moves (Actions) From any given configuration, we locate the blank tile in row r , column c :

$$(r, c) = \{ (i, j) \mid S_{ij} = 0 \}.$$

There are up to four possible actions (**Up**, **Down**, **Left**, **Right**), provided the blank is not already at an edge. Specifically:

- **Up** if $r > 1$: swap $S_{r,c}$ with $S_{r-1,c}$.
- **Down** if $r < 3$: swap $S_{r,c}$ with $S_{r+1,c}$.
- **Left** if $c > 1$: swap $S_{r,c}$ with $S_{r,c-1}$.
- **Right** if $c < 3$: swap $S_{r,c}$ with $S_{r,c+1}$.

Each such swap produces a new successor state. *Each move counts as a unit cost* $g(n) = 1$.

3. Iterative Deepening Scheme One can use **iterative deepening depth-first search** (ID-DFS). The idea is:

1. For depth limit $d = 0, 1, 2, \dots$ (increment d each iteration),
2. Perform a depth-limited DFS from S_{init} with limit d .
3. If we find S_{goal} within depth d , *stop* and return the path.
4. Otherwise, increase d and repeat.

Since we expand all states at depth $\leq d$ on each iteration, the solution path found is guaranteed to be of minimum cost (in terms of number of moves).

4. Pseudocode Below is an example pseudocode that outlines the ID-DFS procedure. We include two main routines:

- **IterativeDeepeningSearch** (the outer loop over depth).
- **DepthLimitedDFS** (the recursive depth-first search with a limit).

Listing 1: Iterative Deepening Depth-First Search for 8-Puzzle

```
function ITERATIVE-DEEPENING-SEARCH(S_init, S_goal):
  for depth_limit in range(0, infinity):
    # Attempt to find goal by searching up to depth_limit
    path <- DEPTH-LIMITED-DFS(S_init, S_goal, depth_limit)
    if path != failure:
      return path # Found a path (solution)
```



```

# If not found (theoretically never ends if unsolvable)
return failure

function DEPTH-LIMITED-DFS(S, S_goal, limit):
  if S == S_goal:
    return [S] # Return the path containing only the goal
  else if limit == 0:
    return failure
  else:
    # Expand children up to depth limit - 1
    for each action in possible_moves(S):
      S_next <- APPLY_ACTION(S, action)
      result <- DEPTH-LIMITED-DFS(S_next, S_goal, limit - 1)
      if result != failure:
        return [S] + result
    return failure

```

Notes on Implementation Details.

- `possible_moves(S)` inspects the blank tile's location and returns up to 4 valid actions (Up, Down, Left, Right).
- `APPLY_ACTION(S, action)` creates a new 3×3 matrix with the blank swapped in the appropriate direction.
- Checking $S == S_{\text{goal}}$ means checking each S_{ij} matches the goal matrix.

5. Condition to Detect the Goal State We compare the current state S at each node with the goal matrix S_{goal} . If all nine tiles match (i.e., $S_{ij} = S_{\text{goal},ij}$ for $1 \leq i, j \leq 3$), we have found a solution.

Conclusion This algorithm will systematically increase the search depth until it finds a sequence of moves that transforms S_{init} into S_{goal} . Iterative deepening combines the space efficiency of depth-first search with the optimality of breadth-first search, ensuring that if a solution exists, we will find the least-cost path where each slide has cost 1. Hence $g(n)$ is just the length of the path (number of blank moves).

(b) First Three Expansions Under Iterative Deepening

Depth limit = 0. We check if S_{init} is already the goal. It is not, so no expansions occur.

Depth limit = 1. (First Expansion) We now allow paths of length ≤ 1 . We *expand* the *initial* node S_{init} (this is our **first expansion**) to generate its children. Since the blank is in position (2,1), there are three possible moves: *Up*, *Down*, and *Right*.

- **Move blank Up:**

$$S_{\text{up}} = \begin{array}{ccc} \square & 5 & 7 \\ 1 & 4 & 2 \\ 8 & 3 & 6 \end{array}$$

- **Move blank Down:**

$$S_{\text{down}} = \begin{array}{ccc} 1 & 5 & 7 \\ 8 & 4 & 2 \\ \square & 3 & 6 \end{array}$$

- **Move blank Right:**

$$S_{\text{right}} = \begin{array}{ccc} 1 & 5 & 7 \\ 4 & \square & 2 \\ 8 & 3 & 6 \end{array}$$

None of these is the goal. Since our limit is 1, we stop searching deeper on this iteration.

Depth limit = 2. Because we did not find a solution at depth 1, we increase the limit to 2 and run a depth-limited DFS again. ID-DFS expands nodes in a *depth-first* manner, so we first expand S_{up} , then S_{down} , and then S_{right} (or any fixed order we choose).

Second Expansion: Expanding S_{up}

We call this the **second expansion** in the search.

$$S_{\text{up}} = \begin{array}{ccc} \square & 5 & 7 \\ 1 & 4 & 2 \\ 8 & 3 & 6 \end{array}$$

The blank is now at position (1, 1). Possible moves:

- *Down* swaps with the tile '1' beneath it, returning us to S_{init} :

$$\begin{array}{ccc} 1 & 5 & 7 \\ \square & 4 & 2 \\ 8 & 3 & 6 \end{array} \quad (\text{which is } S_{\text{init}}).$$

- *Right* swaps with the tile '5':

$$S_{\text{up-right}} = \begin{array}{ccc} 5 & \square & 7 \\ 1 & 4 & 2 \\ 8 & 3 & 6 \end{array}$$

Neither resulting child is the goal.

Third Expansion: Expanding S_{down}

Next (in a typical DFS ordering), we expand

$$S_{\text{down}} = \begin{array}{ccc} 1 & 5 & 7 \\ 8 & 4 & 2 \\ \square & 3 & 6 \end{array}$$

The blank is at position (3,1). Its possible moves are *Up* (swap with '8') and *Right* (swap with '3'), yielding two successors:

$$S_{\text{down-up}} = \begin{array}{ccc} 1 & 5 & 7 \\ \square & 4 & 2 \\ 8 & 3 & 6 \end{array} \quad (\text{again } S_{\text{init}}),$$

and

$$S_{\text{down-right}} = \begin{array}{ccc} & 1 & 5 & 7 \\ 8 & & 4 & 2 \\ 3 & \square & & 6 \end{array}.$$

That completes our **third expansion**.

Summary of the First Three Expansions

Expansion 1: Expand S_{init} (depth limit=1). $\rightarrow \{S_{\text{up}}, S_{\text{down}}, S_{\text{right}}\}$.

Expansion 2: Expand S_{up} (now depth limit=2). $\rightarrow \{S_{\text{init}}, S_{\text{up-right}}\}$.

Expansion 3: Expand S_{down} (depth limit=2). $\rightarrow \{S_{\text{init}}, S_{\text{down-right}}\}$.

If we continued, the next step (fourth expansion) would expand S_{right} , and so on, until eventually finding a path to S_{goal} .

3. Consider a local search problem where the *state space* is a one-dimensional integer $x \in \{0, 1, 2, 3, 4, 5, 6\}$, and the *cost* of a state x is given by $\text{cost}(x) = (x - 3)^2 + 2$. Assume we start at $x = 1$, and at each step we may move to a *neighboring* state $x - 1$ or $x + 1$ (if valid) chosen *at random* with equal probability. We use a *Simulated Annealing* approach with an initial “temperature” T_0 , which is gradually decreased at each iteration by a factor α (e.g. $T \leftarrow \alpha T$).
- (a) Formulate the *acceptance probability* p when the new state x_{next} has a *higher* cost than x_{current} (i.e. $\Delta C > 0$). **(3 Marks)**
- (b) Draw the plot of discrete cost function $\text{cost}(x)$ against the states. **(2 Marks)**
- (c) Suppose we do only two iterations. Let $x_{\text{current}} = 1$ and $\Delta C = \text{cost}(2) - \text{cost}(1)$. With $T = 8$ at the first iteration and $T = 2$ at the second iteration. *Calculate* the *ratio* of the acceptance probabilities of moving from $x = 1$ to $x = 2$ if it is a worse neighbor. *State* the numeric values you get. (**Hint:** $e^{-1.875} = 0.15$) **(5 Marks)**

Q3. Solution & Explanation We have a one-dimensional state space $x \in \{0, 1, 2, 3, 4, 5, 6\}$ with $\text{cost}(x) = (x - 3)^2 + 2$. A Simulated-Annealing (SA) algorithm chooses moves to neighboring states ($x - 1$ or $x + 1$, if valid) at random, then *accepts* or *rejects* a move according to the usual Metropolis criterion based on the “temperature” T .

(a) Acceptance Probability Let x_{current} be the current state and x_{next} the proposed new state. Let

$$\Delta C = \text{cost}(x_{\text{next}}) - \text{cost}(x_{\text{current}}).$$

Under Simulated Annealing, the acceptance probability p is given by:

$$p = \begin{cases} 1, & \text{if } \Delta C \leq 0 \quad (\text{the move is better or equal in cost}), \\ \exp(-\Delta C/T), & \text{if } \Delta C > 0 \quad (\text{the move is worse}). \end{cases}$$

(b) **Plot of cost(x)** Since $\text{cost}(x) = (x - 3)^2 + 2$, we can tabulate:

x	0	1	2	3	4	5	6
$\text{cost}(x)$	11	6	3	2	3	6	11

This is a discrete parabola (minimum at $x = 3$). A rough sketch shows the cost decreasing from $x = 0$ to $x = 3$ and then rising symmetrically up to $x = 6$. Therefore, the curve will look 'U'-shaped.

(c) **Numerical Example with Two Iterations** Suppose we do only two iterations of SA as $T_1 = 8$ (first iteration), $T_2 = 2$ (second iteration). If we start at $x_{\text{current}} = 1$, and consider moving to $x_{\text{next}} = 2$. The difference in cost is

$$\Delta C = \text{cost}(2) - \text{cost}(1) = 3 - 6 = -3.$$

In reality, $\Delta C < 0$, so the move from 1 to 2 is actually *better* and would be accepted with probability 1. However, the question specifically asks us to *imagine* that moving from $x = 1$ to $x = 2$ is a *worse* neighbor (i.e. $\Delta C > 0$ for the sake of illustration). Let us assume some positive ΔC as reverse signed (i.e., 3)

$$p_{\text{first}} = \exp\left(-\frac{\Delta C}{T_1}\right) \quad \text{and} \quad p_{\text{second}} = \exp\left(-\frac{\Delta C}{T_2}\right).$$

With $\Delta C = 3$, $T_1 = 8$, and $T_2 = 2$, we get $p_{\text{first}} = \exp\left(-\frac{3}{8}\right)$, and $p_{\text{second}} = \exp\left(-\frac{3}{2}\right)$. Therefore the ratio is

$$e^{(-3/8)+(3/2)} = e^{9/2}.$$

4. **Context** “Alice’s father, Bob, re-married a widow, Carol, who already had an adult son, David. Now, David is Alice’s stepbrother, and Bob has also legally adopted him. Soon after, Carol had another child, Eve, who is half-sibling to both David *and* Alice—but from different sides.”

(a) Define appropriate *predicate symbols* (e.g. $M(x, y)$ for “ x is married to y ,” $C(x, y)$ for “ x is a child of y ,” etc.). Then encode the main *facts* from the paragraph into *First-Order Logic* formulas. The FoL must capture the following: **(6 Marks)**

- The “stepbrother” relationship or how it arises from “marriage + partial sibling-hood,”
- Bob adopting David,
- Eve’s half-sibling status to David and Alice.

(b) From the previously defined set of FoL formulas, *derive* why David can simultaneously be called Alice’s *stepbrother* and (*legally*) *adopted brother*. Suggest any FOL axiom or short inference chain that demonstrates how “marriage” and “adoption” lead to this relationship. (*Hint*: You may define an “AdoptedSibling(x, y)” or incorporate it into $C(x, y)$ with an extra condition.) **(4 Marks)**

Q4. Solution and Explanation

(a) **Defining Predicates and Encoding the Facts**

- $M(x, y)$: “ x is married to y ” (symmetric).
- $C(x, y)$: “ x is a (legal) child of y ” (may include adoption).
- $\text{Adopt}(p, c)$: “ p has legally adopted c .”

- (Optional) $\text{male}(x)$, $\text{female}(x)$, if needed to distinguish brother/sister.
- $\text{StepBrother}(x, y)$: “ x is the stepbrother of y .”
- $\text{HalfSibling}(x, y)$: “ x is a half-sibling of y .”

Facts from the story:

1. Bob is Alice’s father (so Alice is a child of Bob): $C(\text{Alice}, \text{Bob})$.
2. Bob remarried Carol (a widow): $M(\text{Bob}, \text{Carol})$.
3. Carol already had an adult son, David: $C(\text{David}, \text{Carol})$.
4. David is Alice’s stepbrother (from marriage + partial siblinghood). We can capture it either by an explicit statement $(\text{David}, \text{Alice})$, or by a *definitional* axiom such as

$$\forall x, y \left[\text{StepBrother}(x, y) \leftrightarrow \text{male}(x) \wedge \exists p, q \left(M(p, q) \wedge p \neq q \wedge C(x, p) \wedge C(y, q) \right) \right].$$

In words: x is a stepbrother of y if x is male and x ’s parent p is married to y ’s parent q , but $p \neq q$.

5. Bob has legally adopted David: $\text{Adopt}(\text{Bob}, \text{David})$. An additional axiom can say adopting makes one into the (legal) parent: $\forall p, c : \text{Adopt}(p, c) \rightarrow C(c, p)$.
6. Carol and Bob then have another child, Eve: $C(\text{Eve}, \text{Carol})$, $C(\text{Eve}, \text{Bob})$.
7. Eve is the half-sibling of both David and Alice. A common formalization is

$$\forall x, y : \text{HalfSibling}(x, y) \leftrightarrow (\exists p : C(x, p) \wedge C(y, p)) \wedge (\text{they share exactly one parent}).$$

Concretely, $\text{HalfSibling}(\text{Eve}, \text{Alice})$, $\text{HalfSibling}(\text{Eve}, \text{David})$. This encodes that Eve shares exactly one parent with David (Carol) and exactly one parent with Alice (Bob).

(b) Deriving David’s Dual Status (Stepbrother + Adopted Brother)

Key reasoning steps:

- From $M(\text{Bob}, \text{Carol})$, $C(\text{Alice}, \text{Bob})$, and $C(\text{David}, \text{Carol})$, we infer that David is Alice’s *stepbrother*. Intuitively, David and Alice do not share *both* parents, but one parent of Alice (Bob) is married to one parent of David (Carol).
- From $\text{Adopt}(\text{Bob}, \text{David})$ and the axiom $\text{Adopt}(p, c) \rightarrow C(c, p)$, we conclude $C(\text{David}, \text{Bob})$ in the legal sense. Consequently, David and Alice *also* share a parent (Bob), making them legal siblings (brother/sister).

Hence David can be both:

1. *Stepbrother* of Alice (through Carol’s marriage to Bob when David was Carol’s son)
2. *Adopted brother* of Alice (because Bob adopted David, so David is also legally Bob’s child).

This satisfies the narrative that someone can be a *step-sibling* (due to parental re-marriage) yet also an *adopted sibling* (due to formal adoption by the step-parent).